

PSTeC: A Location-Time Driven Modelling Formalism for Probabilistic Real-Time Systems

Kangli He¹, Yixiang Chen^{1(✉)}, Min Zhang², and Yuanrui Zhang¹

¹ MoE Engineering Research Center for Software/Hardware Co-design Technology and Application, East China Normal University, Shanghai, China

kenhkl11@hotmail.com, yxchen@sei.ecnu.edu.cn

² Shanghai Key Laboratory of Trustworthy Computing,
East China Normal University, Shanghai, China

mzhang@sei.ecnu.edu.cn

Abstract. Internet of Things (IoT) and Cyber-Physical Systems (CPS) have become important topics in both theory and industry. In some application domains, such as when specifying the behaviour of precision mechanics, we need to include features of spatial-temporal consistency. How to model probabilistic real-time systems in such domains is a challenge. This paper presents a modelling formalism, called PSTeC, for describing the behaviour of probabilistic real-time systems focusing on spatial-temporal consistency with nondeterministic, probabilistic and real-time aspects. The consistency restricts a process to start and finish at the required location and time. Communications between agents is specified by interactive actions. The language we propose is an extension of STeC, which is a specification language for location-aware real-time systems, adding probabilistic operations so as to support the incorporation of probabilistic aspects. We first give a formal definition of the syntax for PSTeC, then focus on the details of its operational semantics, which maps a PSTeC term onto a Probabilistic Spatial-Temporal Transition System (PSTTS) following the structured operational semantics style. A simple example demonstrates the expressiveness of PSTeC.

1 Introduction

Real-time systems are everywhere, spreading from tiny micro chips to continent-spanning power grids. Internet of Things and Cyber-Physical Systems are *en vogue* incarnations of real-time systems featuring nondeterministic, probabilistic and real-time aspects. Often, we need to consider not only the time aspect but also the location of one agent, i.e., we want the behaviour of an agent to start and finish at the required location and time. During the system design phase, we usually use formal methods to model the system and then check whether requirements on correctness do hold. If that is not the case, one tries to fix the model so as to eventually guarantee the absence of errors before beginning implementation. Chen [2] proposes a location-triggered specification language for real-time systems, called STeC, which emphasizes *location*, *time*, and especially on the

consistency between them. The *location* describes the physical location (e.g., a particular bus station in Münster, Germany) or a state (e.g., “closed”) of agents, the *time* focuses on exact physical time (e.g., 5:22pm) at which the behaviour starts or finishes, together with the duration expressing how long the behaviour executes or persists. Intuitively, we aim at allowing such a process to take place and finish at the required location and time, and with the specified execution time, so as to guarantee the spatial-temporal *consistency*. This is very important to ensure the success of tasks in real-time systems concerning precision, such as railway crossing systems or robot hands. The syntax of STeC [2] resembles some classical process algebras, such as CSP and CCS, it uses processes to describe the behaviours of agents and systems, but STeC restricts each atomic or compositional process by a pair of (starting) location and time, and also a duration, i.e., all the processes are triggered by location and time.

STeC has been extended with a hybrid clock to specify both logical and chronometric time aspects of real-time system [4], and is powerful enough for real-time systems, yet it does not support the description or specification of probabilistic aspect as they prevail in probabilistic real-time systems. In this paper, we address this challenge. Formalization of probabilistic systems has become an important topic of investigations in theoretical computer science. Segala [16] defines several probabilistic models and simulation relations for probabilistic processes, and He [10] has extended the Dijkstra’s guarded command languages [5] with a probabilistic choice operator to express randomized algorithms. Ying [19] develops formal methods and mathematical tools for modeling and reasoning about programs containing probability information, and Chen [3] has extended probabilistic choice in probabilistic programs to sub-probabilistic choice. In recent decades, model checking methods for probabilistic and probabilistic timed automata have been developed, and implemented among others by Kwiatkowska et al. [13, 14]. Hermanns et al. [7, 9, 11] have developed various model checkers for probabilistic systems, and He [8] uses the probabilistic model checker PRISM [12] to verify a communication protocol under a specific Internet of Things (called WInternet).

As mentioned above, the modelling mechanisms for probabilistic systems are not new, but lack of a formalism for precision real-time systems, in which location and time are the two most important keys to success for tasks. Modern languages, such as Modest [1] and the PRISM language (i.e., guarded commands), are easy for users to learn and are capable to express a probabilistic real-time system model focusing on time, not on locality. This means that the model is driven by (discrete or continuous) time without considering the physical location of each agent. Location and the spatial-temporal consistency are not considered of interest. In contrast, PSTeC not only deals with the basic modelling of probabilistic, nondeterministic and real-time characteristics, but also takes location and spatial-temporal consistency aspects into account. PSTeC is also an easy-to-learn language for non-scientist users such as engineers. Inspired by the general approach to reason about probabilistic systems, we extend the syntax of STeC with additional probabilistic operators equipped with an operational semantics.

This has already proved successful for non-probabilistic real-time system, with the inclusion of probabilistic distributions. Notably, adding a probabilistic feature to the STeC syntax will change the original semantics in its entirety (i.e., not just simply appending additional semantics for the probabilistic features), and also we need to find a suitable way to integrate the features semantically. To do so, we take inspiration from the way the operational semantics of Modest [1, 6] lifts structural operational semantics to work with probabilistic distributions over complex structures.

Organisation of the Paper. Section 2 first defines the syntax of PSTeC formally with some useful shorthands to help modelling, and then develops the operational semantics of PSTeC in full detail. A model of a smart spray painting factory is presented as a case in Sect. 3 to show how to use PSTeC to specify location-aware probabilistic real-time systems. Finally, Sect. 4 concludes the paper.

2 Formal Definition of PSTeC

In this section, we present the formalism of PSTeC, as an extension of STeC [2]. To be clear with respect to the description of behaviours of individual **Agent**, respectively the concurrent **System**, we give the definition of PSTeC syntax for agents and systems separately. Processes P for **Agent** and S for **System** are constructed according to the grammar given in Eqs. 1 and 2 respectively.

2.1 Syntax

(1) For each **Agent**:

$$\begin{aligned}
 A &::= \text{Send}_{(l,t)}^{\leftarrow G'}(m, \delta) \mid \text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta) \\
 B &::= \text{Act}_{(l,t)}(l', \delta) \mid \text{Stat}_{(l,t)}(\delta) \\
 C &::= A \mid B \\
 P &::= \text{Stop}_{(l,t)}(\infty) \mid \text{Skip}_{(l,t)}(0) \mid C \mid \\
 &\quad P; P \mid P \parallel P \mid P \parallel B \rightarrow P \mid \\
 &\quad P \triangleright_{\delta} P \mid P \triangleright \parallel_{i \in I} \text{Get}_{(l,t)}^{\leftarrow G'}(m_i, \delta) \rightarrow P_i \mid \\
 &\quad C \rightarrow +_{i \in I} p_i : \{as_i\} P_i
 \end{aligned} \tag{1}$$

In order to track and record the messages concerned in the system, we define a *storage* for each agent, denoted by $\sigma \subseteq 2^{\mathcal{M}}$ (if necessary, we use $\sigma_{G'}$ to denote the *storage* for agent G'), where $\mathcal{M}$ is the set of messages with the form $m_{G \uparrow G'}$ or m , and $\sum$ as the set of storages (i.e., $\sigma \in \sum$). Note that, $m_{G \uparrow G'}$ represents the message from G to G' , and we use $\perp$ to describe unknown agent, i.e., $m_{\perp \uparrow G}$ stands for the message from some unknown agent to agent G . The duration δ takes the non-negative real numbers as well as the infinite ∞ . If δ takes the ∞ value then it means the process will continue to execute and will not stop, and if δ is 0 then it means that the execution of the process takes no time.

Atomic Processes. We assume that the set of atomic processes $\mathcal{AP}$ are partitioned into a set of *interactions* $\mathcal{A}$, a set of *guards* $\mathcal{B}$, Stop and Skip.

Two interaction processes Send and Get are involved in our language, they are used to describe the communication between two agents through messages without changing locations. Informally, the atomic process $\text{Send}_{(l,t)}^{\rightarrow G'}(m, \delta)$ shows that agent G sends message m (i.e., $m_{G \rightarrow G'}$) to the destination agent G' at location l and at time t , taking time δ . Similarly, agent G executes $\text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta)$ to get message m (i.e., $m_{G' \rightarrow G}$) from agent G' by δ duration.

Two most common atomic processes are action Act and state Stat, which are also called *guards* of the Dijkstra's guarded type denoted by B . $\text{Act}_{(l,t)}(l', \delta)$ represents that the action Act is executed at location l and at time t , taking δ duration, when the action finishes, the agent arrives at location l' . $\text{Stat}_{(l,t)}(\delta)$ represents that at location l and at time t , the agent keeps state Stat for δ unit times. Note that, l and l' in B can either represent the location (e.g., *Lapp*) or state (e.g., *Open*) according to different real situations. The *guard* B takes the Boolean/truth values: True T and False F . We use $\mathcal{B}$ to denote the set of guards, i.e., $B \in \mathcal{B}$, then a *truth assignment* for guard B is a function $\nu : \mathcal{B} \rightarrow \{T, F\}$, if action $\text{Act}_{(l,t)}(l', \delta)$ finishes within the δ time units and moves to location l' , then $\nu(\text{Act}_{(l,t)}(l', \delta)) = T$ (i.e., True), else $\nu(\text{Act}_{(l,t)}(l', \delta)) = F$ (i.e., False). Similarly, if state $\text{Stat}_{(l,t)}(\delta)$ keeps δ time units, then $\nu(\text{Stat}_{(l,t)}(\delta)) = T$, else $\nu(\text{Stat}_{(l,t)}(\delta)) = F$. Typically, the valuation **False** formally specify such situation: (1) Action $\text{Act}_{(l,t)}(l', \delta)$ finishes less than or more than δ duration. (2) State $\text{Stat}_{(l,t)}(\delta)$ lasts less than or more than δ time units. 3) When action $\text{Act}_{(l,t)}(l', \delta)$ or state $\text{Stat}_{(l,t)}(\delta)$ finishes, the agent is not at location l' .

Process $\text{Stop}_{(l,t)}(\infty)$ will never finish unless it is interrupted. $\text{Skip}_{(l,t)}(0)$ will finish immediately without taking any time.

Compositional Processes. $P; P$ is the sequential composition. $P_1 \parallel P_2$ represents the parallel, since we only consider the handshake above Send-Get interactions and there is no possible that same message m is sent and got by agent G at the same time, P_1 and P_2 just behave independently. Note that because we have to restrict each atomic process with the triggered location and time as well as the execution duration, so P_1 and P_2 will behave following their own procedure instead of interleaving. $P \parallel P$ represents nondeterministic choice and executes if one of the sub processes executes. $B \rightarrow P$ stands for the If-Then statement that if guard B is True (i.e., $\nu(B) = T$) then agent behaves like process P .

Timeout process $P_1 \supseteq_{\delta} P_2$ behaves as P_1 for δ time units and then behaves as P_2 , in another word, process P_1 is interrupted by duration δ . $P \supseteq \parallel_{i \in I} \text{Get}_{(l,t)}^{\leftarrow G'}(m_i, \delta) \rightarrow P_i$ is another type of interruption that P executes until agent G finishes Get, which starts at location l and at time t , and gets message m_i , then agent behaves like process P_i .

$C \rightarrow +_{i \in I} p_i : \{as_i\} P_i$, which makes the main difference from STeC to PSTeC, represents a probabilistic choice that after C is executed the process behaves as P_i with probability p_i , where C can be arbitrary atomic process except Stop and Skip. p_i denotes the probability measure for process P_i and $\sum_{i \in I} p_i = 1$.

as_i represents a set of assignments with the form: $x_1 = e_1, \dots, x_n = e_n$, where $x_i \in \text{Var}$ ($0 < i \leq n$) are different variables and e_i can be constants (e.g., 5, *closed*), variables (e.g., x_1), or arithmetic expressions (e.g., $x_1 - 5$). The set of as_i is denoted by $\mathbb{A}$. Similarly, we use $\mathcal{C}$ to denote the set of C , and have the *truth assignment* for *probabilistic guard* C , i.e., a function $\rho : \mathcal{C} \rightarrow \{T, F\}$. The truth assignment for the subset $\mathcal{B} \subseteq \mathcal{C}$ is the same as that described in Sect. 2.1. Now we only consider the part composed by A : If action $\text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta)$ finishes successfully within δ time units and there exists m (specifically $m_{G' \uparrow G}$) in the storage of G' (i.e., $m_{G' \uparrow G} \in \sigma_{G'}$), $\rho(\text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta)) = T$ (i.e., True). If action $\text{Send}_{(l,t)}^{\rightarrow G'}(m, \delta)$ finishes successfully within δ time units and m (specifically $m_{G \uparrow G'}$) exists in the storage of G (i.e., $m_{G \uparrow G'} \in \sigma_G$), $\rho(\text{Send}_{(l,t)}^{\rightarrow G'}(m, \delta)) = T$ (i.e., True). Considering probabilistic choice process:

$$\begin{aligned} & \text{Closing}_{(L,t)}(L, \theta_1) \rightarrow \\ & \left(\begin{array}{l} 0.8 : \{st = \text{closed}\}(\text{Send}_{(L,t+\theta_1)}^{\rightarrow T}(CR, 2) \parallel \text{Closed}_{(L,t+\theta_1)}(\theta_2)) \\ + \\ 0.2 : \{st = \text{nclosed}\}(\text{Send}_{(L,t+\theta_1)}^{\rightarrow T}(NC, 2) \parallel \text{Closing}_{(L,t+\theta_1)}(L, \theta_3)) \end{array} \right) \end{aligned}$$

where the variable st stands for the state of **Gate** with initial value *open*. **Gate** takes action **Closing** to try to close itself, and when this action is done after θ_1 time units: (1) The state of **Gate** is closed with probability 0.8, then **Gate** sends message *CROSS* to **Train** and meanwhile it stays Closed. Or (2) the state is non-closed with 0.2, then **Gate** sends message *NONCROSS* to **Train** and meanwhile it continues to close.

(2) For the **System**:

$$S ::= P_G \mid S \bowtie S \quad (2)$$

The system process consists of $n(\geq 1)$ agent processes behaving concurrently (i.e., $P_{G_1} \bowtie P_{G_2} \bowtie \dots \bowtie P_{G_n}$). If there are two different agents G_1 and G_2 in the system, then they handshake on interactions (i.e., if $\text{Send}_{(l_{G_1}, t_{G_1})}^{\rightarrow G_2}(m_{G_1 \uparrow G_2}, \delta_{G_1})$ and $\text{Get}_{(l_{G_2}, t_{G_2})}^{\leftarrow G_1}(m_{G_1 \uparrow G_2}, \delta_{G_2})$ are ingredients of P_{G_1} and P_{G_2} respectively, and $t_{G_1} = t_{G_2}$, $\delta_{G_1} = \delta_{G_2}$, then P_{G_1} and P_{G_2} should execute the above interactions at the same time), otherwise they behave independently. Similarly, for n agents, if $\text{Send}^{\rightarrow G_i}$ and $\text{Get}^{\leftarrow G_j}$ with the same message (note that we use $m_{G_i \uparrow G_j}^k$ to represent $m_{G_i \uparrow G_j}$ from G_i to G_j at k th time, where $k \geq 1$, which means different time-stamped messages from the same sender and receiver G_i to G_j) exist in sub processes P_{G_j} and P_{G_i} respectively, then the above restrictions should be guaranteed, otherwise they behave independently.

We use $\mathcal{P}$ to denote the set of agent processes (i.e., $P \in \mathcal{P}$) and $\mathcal{S}$ as the set of system processes (i.e., $S \in \mathcal{S}$). Notation E is used to describe a successful termination, and considering P we always rewrite processes of form $E; P$, $P; E$, $E \parallel P$, $P \parallel E$ as P . The extended set of processes are denoted by $\mathcal{P}^E = \mathcal{P} \cup E$. Note that the structure $(\mathcal{P}^E, E, ;, \parallel)$ is a bimonoid, and $(\mathcal{P}^E, E, \parallel)$ is a commutative monoid.

Reed and Roscoe [17] defined a process $\text{WAIT } t$ to specify the waiting state lasting for t time units. Same as in [2], we define agent process Wait as:

$$\text{Wait}_{(l,t)}(\delta) \stackrel{\text{def}}{=} \text{Stop}_{(l,t)}(\infty) \supseteq_{\delta} \text{Skip}_{l,t+\delta}(0) \quad (3)$$

to describe agent G waits for δ time units and does nothing, and

$$\text{Wait}_{(l,t)}^{\leftarrow G'}(m) \stackrel{\text{def}}{=} \text{Stop}_{(l,t)}(\infty) \supseteq \text{Get}_{(l,t')}^{\leftarrow G'}(m, \delta) \rightarrow \text{Skip} \quad (4)$$

to describe agent G waits for the message m coming from G' without changing the location, where $t < t'$.

2.2 Some Constraints

Since giving the syntax of PSTeC, we need to clarify some constraints that ensure the statements are meaningful and practical.

1. For $P_1; P_2$, $P_1 \parallel P_2$ and $P_1 \parallel P_2$, the starting location and time of P_2 should be the same as those of P_1 respectively.
2. For $P = P_1 \parallel P_2$, If P has successive processes, the finishing location and time of P_1 and P_2 are the same respectively.

2.3 Operational Semantics

An *environment* e for each **Agent** is a triple (a, u, σ) , where (a, u) is a *spatial-temporal point*, in which a stands for the location or state, and u is a global clock in the system. We use $\mathcal{E} : \mathcal{L} \times \mathcal{T} \times \Sigma$ to denote the set of environments, where $\mathcal{L}$ represents the set of location variables and $\mathcal{T}$ represents the set of time variables. We define functions $t(P) : \mathcal{P} \rightarrow \mathcal{T}$ to extract the triggered time ingredient of process P and $l(P) : \mathcal{P} \rightarrow \mathcal{L}$ to extract the triggered location ingredient of process P . Based on the environment (a, u, σ) , we give the *truth assignment* for the probabilistic guards C as a function $\mathfrak{T} : \mathcal{C} \times \mathcal{E} \rightarrow \{T, F\}$. $\mathfrak{T}(C)(a, u, \sigma) = T$ if and only if $(l(C), t(C)) = (a, u)$ and $\rho(C) = T$.

We define the set of configurations **Stconf** as $\mathcal{P}^E \times \mathcal{E}$. A configuration is of the form $\langle P, (a, u, \sigma) \rangle$ denoted by π , where P is a agent process or E . The operational semantics for agent process P is given in terms of the *probabilistic spatial-temporal transition system* following the structured operational semantics style (SOS) [15], in which the operational semantic is described as the individual step of its operands. To handle the probabilistic aspects (induced by the probabilistic choice operator, but affecting the entirety of the semantics) we take inspirations from the Modest operational semantics [1, 6].

We define *spatial-temporal probability measure* (st-probability measure for short) as a function $\mathbf{P} : \mathbb{A} \times \mathbf{Stconf} \rightarrow [0, 1]$ to map an assignment and a configuration onto a probability.

Definition 1. A probabilistic spatial-temporal transition system (*PSTTS*) is defined as a subset $\rightarrow \subseteq \mathbf{Stconf} \times \mathbf{P}(\mathbb{A} \times \mathbf{Stconf})$.

where $\mathbf{P}$ is a kind of “symbolic” probabilistic distribution over pairs of assignments and target configurations. Intuitively speaking, for $\langle \pi, \mathbf{P}(as_i, \pi_i) \rangle \in \rightarrow$, we write $\pi \rightarrow \mathbf{P}(as_i, \pi_i)$, and once $\pi \rightarrow \mathbf{P}(as_i, \pi_i)$ is executed, the configuration is changed to π_i assigning variables values according to as_i with probability $\mathbf{P}(as_i, \pi_i)$, satisfying for all target pairs of (as_i, π_i) that $\sum_{i \in I} \mathbf{P}(as_i, \pi_i) = 1$.

Let $\mathcal{D}(as, \pi)$ denote the *deterministic* (often called *Dirac*) probability measure that is defined by $\mathcal{D}(as, \pi)(as, \pi) = 1$ and $\mathcal{D}(as, \pi)(as', \pi') = 0$ for all $(as', \pi') \neq (as, \pi)$. Intuitively, the assignment as and configuration π are chosen with probability 1.

Before presenting the operational semantics, we first define some functions. $\iota(P), \tau(P), \kappa(P)$ are extended from [18]: $\iota(P)$ denotes the location where P is finished, $\tau(P)$ denotes the duration that P is executed successfully, and $\kappa(P)$ records the store of messages after process P finished. We use $\ominus(P)(a, u, \sigma) : \mathcal{P} \times \mathcal{L} \times \mathcal{T} \times \mathcal{M} \rightarrow \mathcal{M}$ to record the set of messages that are going to be removed from σ after the execution of process P under the environment (a, u, σ) . Function Last is defined to find the max location (which can only be comparable with the same agent). $\iota(P)^{t+\delta}$ and $\kappa(P)^{t+\delta}$ record the location and storage of process P at time $(t + \delta)$ respectively. $\ominus(P)^{t+\delta}$ records the set of messages that are produced at time $(t + \delta)$ in $\ominus(P)$. These functions will be used in the following subsections.

Time Function. The execution time $\tau(P)$ of a process P is defined in Table 1, where ¹: **if** the process has no successive process we simply omit the time of this process since we really don not care under this condition, **else** we restrict $\tau(P_1) = \tau(P_2)$, **so** ¹ = $\tau(P_1)$, ²: similarly **if** there is no successors we omit it **else** we restrict $\tau(P_i) = \tau(P_{i+1})$, **so** ² = $t - t(P) + \delta + \tau(P_i)$.

Location Function. We define the location function $\iota(P)$ under the environment (l, t, σ) as in Table 2, where ³: **if** the process has no successor we omit it **else** we restrict $\iota(P_1) = \iota(P_2)$, **so** ³ = $\iota(P_1)$, ⁴: **if** no successor we omit it **else** we restrict $\iota(P_1) = \iota(P_2)$, **so** ⁴ = $\iota(P_i)(\iota(P)^{t'+\delta}(l, t, \sigma), t' + \delta, \kappa(P)^{t'+\delta}(l, t, \sigma) \cup \{m_{G' \uparrow G}\})$, ⁵: = $\text{Last}\{\iota(P_i)(\iota(C)(l, t, \sigma), t + \tau(C), \kappa(C)(l, t, \sigma))\} \mid i \in I$.

Message Function. The message function $\kappa(P)$ under the environment (l, t, σ) is defined in the Table 3, where ⁶: notice that, we allow message redundancy, i.e., if the parallel process is chosen to behave as P_1 , then $\kappa(P_1) \subseteq \kappa(P_1 \parallel P_2)$, or if P_2 is chosen, then $\kappa(P_2) \subseteq \kappa(P_1 \parallel P_2)$, ⁷: = $\cup_{i \in I} \kappa(P_i)(\iota(P)^{t'+\delta}(l, t, \sigma), t' + \delta, \kappa(P)^{t'+\delta}(l, t, \sigma) \cup \{m_{G' \uparrow G}\})$.

Now we first present the operational semantics of agent process and then the operational semantics for a more complex system process.

Basic Processes. Behaviour $\text{Stop}_{(l, t+1)}(\infty)$ will not terminate before it is interrupted and $\text{Skip}_{(l, t)}(0)$ terminates immediately, both of them will neither change the value of any variable nor the storage of the agent. When the spatial-temporal

Table 1. Time function $\tau(P)$.

$\tau(\text{Stop}_{(l,t)}(\infty)) = \infty$	$\tau(\text{Skip}_{(l,t)}(0)) = 0$
$\tau(\text{Send}_{(l,t)}^{\rightarrow G'}(m, \delta)) = \delta$	$\tau(\text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta)) = \delta$
$\tau(\text{Act}_{(l,t)}(l', \delta)) = \delta$	$\tau(\text{Stat}_{(l,t)}(\delta)) = \delta$
$\tau(P_1; P_2) = \tau(P_1) + \tau(P_2)$	$\tau(P_1 \parallel P_2) = \max\{\tau(P_1), \tau(P_2)\}$
$\tau(P_1 \parallel P_2)^1$	$\tau(B \rightarrow P) = \tau(B) + \tau(P)$
$\tau(P_1 \triangleright_{\delta} P_2) = \delta + \tau(P_2)$	$\tau(P \triangleright_{i \in I} \text{Get}_{(l,t)}^{\leftarrow G'}(m_i, \delta) \rightarrow P_i)^2$
$\tau(C \rightarrow +_{i \in I} p_i : \{as_i\} P_i) = \tau(C) + \max\{\tau(P_i)\} \mid i \in I$	

Table 2. Location function $\iota(P)$.

$\iota(\text{Stop}_{(l,t)}(\infty))(l, t, \sigma) = l$	$\iota(\text{Skip}_{(l,t)}(0))(l, t, \sigma) = l$
$\iota(\text{Send}_{(l,t)}^{\rightarrow G'}(m, \delta))(l, t, \sigma) = l$	$\iota(\text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta))(l, t, \sigma) = l$
$\iota(\text{Act}_{(l,t)}(l', \delta))(l, t, \sigma) = l'$	$\iota(\text{Stat}_{(l,t)}(\delta))(l, t, \sigma) = l$
$\iota(P_1; P_2)(l, t, \sigma) = \iota(P_2)(\iota(P_1)(l, t, \sigma), t + \tau(P_1), \kappa(P_1)(l, t, \sigma))$	
$\iota(P_1 \parallel P_2)(l, t, \sigma) = \text{Last}\{\iota(P_1)(l, t, \sigma), \iota(P_2)(l, t, \sigma)\}$	
$\iota(P_1 \parallel P_2)(l, t, \sigma)^3$	$\iota(B \rightarrow P) = \iota(P)(\iota(B)(l, t, \sigma), t + \tau(B), \sigma)$
$\iota(P_1 \triangleright_{\delta} P_2)(l, t, \sigma) = \iota(P_2)(\iota(P_1)^{t+\delta}(l, t, \sigma), t + \delta, \kappa(P_1)^{t+\delta}(l, t, \sigma))$	
$\iota(P \triangleright_{i \in I} \text{Get}_{(l',t')}^{\leftarrow G'}(m_i, \delta) \rightarrow P_i)(l, t, \sigma)^4$	$\iota(C \rightarrow +_{i \in I} p_i : \{as_i\} P_i)(l, t, \sigma)^5$

Table 3. Message function $\kappa(P)$.

$\kappa(\text{Stop}_{(l,t)}(\infty))(l, t, \sigma) = \sigma$	$\kappa(\text{Skip}_{(l,t)}(0))(l, t, \sigma) = \sigma$
$\kappa(\text{Send}_{(l,t)}^{\rightarrow G'}(m, \delta))(l, t, \sigma) = \sigma \setminus \{m_{G' \vdash G'}\}$	$\kappa(\text{Get}_{(l,t)}^{\leftarrow G'}(m, \delta))(l, t, \sigma) = \sigma \cup \{m_{G' \vdash G'}\}$
$\kappa(\text{Act}_{(l,t)}(l', \delta))(l, t, \sigma) = \sigma$	$\kappa(\text{Stat}_{(l,t)}(\delta))(l, t, \sigma) = \sigma$
$\kappa(P_1; P_2)(l, t, \sigma) = \kappa(P_2)(\iota(P_1)(l, t, \sigma), t + \tau(P_1), \kappa(P_1)(l, t, \sigma))$	
$\kappa(P_1 \parallel P_2)(l, t, \sigma) = \kappa(P_1)(l, t, \sigma) \cup \kappa(P_2)(l, t, \sigma) \setminus (\ominus(P_1)(l, t, \sigma) \cup \ominus(P_2)(l, t, \sigma))$	
$\kappa(P_1 \parallel P_2)(l, t, \sigma) = \kappa(P_1)(l, t, \sigma) \cup \kappa(P_2)(l, t, \sigma)^6$	
$\kappa(B \rightarrow P) = \kappa(P)(\iota(B)(l, t, \sigma), t + \tau(B), \sigma)$	
$\kappa(P_1 \triangleright_{\delta} P_2)(l, t, \sigma) = \kappa(P_2)(\iota(P_1)^{t+\delta}(l, t, \sigma), t + \delta, \kappa(P_1)^{t+\delta}(l, t, \sigma))$	
$\kappa(P \triangleright_{i \in I} \text{Get}_{(l',t')}^{\leftarrow G'}(m_i, \delta) \rightarrow P_i)(l, t, \sigma)^7$	
$\kappa(C \rightarrow +_{i \in I} p_i : \{as_i\} P_i)(l, t, \sigma) = \cup_{i \in I} \kappa(P_i)(\iota(C)(l, t, \sigma), t + \tau(C), \kappa(C)(l, t, \sigma))$	

point coincides the triggered location and time of the processes, the operational semantics read:

$$\frac{(a, u) = (l, t)}{\langle \text{Stop}_{(l,t)}(\infty), (a, u, \sigma) \rangle \rightarrow \mathcal{D}(\emptyset, \pi)}$$

where $\pi = \langle \text{Stop}_{(l,t)}(\infty), (l, t, \sigma) \rangle$, and

$$\frac{(a, u) = (l, t)}{\langle \text{Skip}_{(l,t)}(0), (a, u, \sigma) \rangle \rightarrow \mathcal{D}(\emptyset, \pi)}$$

where $\pi = \langle E, (l, t, \sigma) \rangle$.

Since the assignments are partial functions, $\emptyset$ stands for the empty assignment that no variable changes its value.

Interactions Send and Get specify the interaction or handshaking between two agents by messages. Send sends a message m (specifically $m_{G \uparrow G'}$) from storage G to the storage of G' . Note that the procedure that m is added into $\sigma_{G'}$ is not presented here but in the operational semantics of Get process in agent G' . Whilst, Get gets a message m (specifically $m_{G' \uparrow G}$) from $\sigma_{G'}$. Similarly, the procedure of judgement for $m_{G' \uparrow G} \in \sigma_{G'}$ is presented in the operational semantics of Send in G . In fact the above two procedures will be presented in the operational semantics of system process in later subsection. No assignments are executed:

$$\frac{(a, u) = (l, t) \wedge m_{G \uparrow G'} \in \sigma}{\langle \text{Send}_{(l,t)}^{-G'}(m, \delta), (a, u, \sigma) \rangle \rightarrow \mathcal{D}(\emptyset, \pi)}$$

where $\pi = \langle E, (l, t + \delta, \sigma \setminus \{m_{G \uparrow G'}\}) \rangle$, and

$$\frac{(a, u) = (l, t)}{\langle \text{Get}_{(l,t)}^{-G'}(m, \delta), (a, u, \sigma) \rangle \rightarrow \mathcal{D}(\emptyset, \pi)}$$

where $\pi = \langle E, (l, t + \delta, \sigma \cup \{m_{G' \uparrow G}\}) \rangle$.

We require that action Act takes δ time units to execute successfully moving to location l' and Stat keeps δ without changing the location. No message are involved and both processes execute no assignments:

$$\frac{(a, u) = (l, t)}{\langle \text{Act}_{(l,t)}(l', \delta), (a, u, \sigma) \rangle \rightarrow \mathcal{D}(\emptyset, \pi)}$$

where $\pi = \langle E, (l', t + \delta, \sigma) \rangle$, and

$$\frac{(a, u) = (l, t)}{\langle \text{Stat}_{(l,t)}(\delta), (a, u, \sigma) \rangle \rightarrow \mathcal{D}(\emptyset, \pi)}$$

where $\pi = \langle E, (l, t + \delta, \sigma) \rangle$.

Sequential Composition. $P_1; P_2$ executes P_1 until it successfully terminates, then it continues with the execution of P_2 . When sequential process executes $\delta_0 \leq \tau(P_1; P_2)$ time units, like in [1], we define the operational semantics as:

$$\frac{\langle P_1, (a, u, \sigma) \rangle \rightarrow \mathbf{P}}{\langle P_1; P_2, (a, u, \sigma) \rangle \rightarrow \mathbf{P} \circ \mathbf{M}_\downarrow^{-1}}$$

where

$$\mathbf{M}_\downarrow(as, \langle P'_1, e \rangle) \stackrel{def}{=} \begin{cases} \langle as, \langle P'_1; P_2, e \rangle \rangle & \text{if } P'_1 \neq E \\ \langle as, \langle P_2, e \rangle \rangle & \text{if } P'_1 = E \end{cases}$$

and $e = (\iota(P_1)^{u+\delta_0}(a, u, \sigma), u + \delta_0, \kappa(P_1)^{u+\delta_0}(a, u, \sigma))$. The inverse of $\mathbf{M}_\downarrow$ is used in $\mathbf{P} \circ \mathbf{M}_\downarrow^{-1}$ to retrieve the st-probability measure for the sequential composition from the st-probability measure assigned by $\mathbf{P}$ to the first component of a sequential composition.

Parallel Composition. As mentioned in Sect. 2.1, parallel composition $P_1 \parallel P_2$ in agent process simply behaves independently:

$$\frac{\langle P_1, (a, u, \sigma) \rangle \rightarrow \mathbf{P}_1 \wedge \langle P_2, (a, u, \sigma) \rangle \rightarrow \mathbf{P}_2}{\langle P_1 \parallel P_2, (a, u, \sigma) \rangle \rightarrow (\mathbf{P}_1 \times \mathbf{P}_2) \circ \mathbf{M}_\parallel^{-1}}$$

where $(\mathbf{P}_1 \times \mathbf{P}_2)(\Omega_1, \Omega_2) \stackrel{def}{=} \mathbf{P}_1(\Omega_1) \cdot \mathbf{P}_2(\Omega_2)$ for all Ω_1 and Ω_2 , corresponding to the product of two probability spaces, and

$$\begin{aligned} & \mathbf{M}_\parallel(\langle as_1, \langle P'_1, e_1 \rangle \rangle, \langle as_2, \langle P'_2, e_2 \rangle \rangle) \\ & \stackrel{def}{=} \begin{cases} \langle as_1 \cup as_2, \langle P'_1 \parallel P'_2, e \rangle \rangle & \text{if } P'_1 \neq E \text{ or } P'_2 \neq E \\ \langle as_1 \cup as_2, \langle E, e \rangle \rangle & \text{if } P'_1 = E \text{ and } P'_2 = E \end{cases} \end{aligned}$$

where $e_1 = (l_1, u + \delta_0, \sigma_1)$, $e_2 = (l_2, u + \delta_0, \sigma_2)$, $e = (\text{Last}\{l_1, l_2\}, u + \delta_0, \sigma_\parallel)$, in which $l_1 = \iota(P_1)^{u+\delta_0}(a, u, \sigma)$, $\sigma_1 = \kappa(P_1)^{u+\delta_0}(a, u, \sigma)$, $l_2 = \iota(P_2)^{u+\delta_0}(a, u, \sigma)$, $\sigma_2 = \kappa(P_2)^{u+\delta_0}(a, u, \sigma)$, $\sigma_\parallel = \sigma_1 \cup \sigma_2 \setminus (\ominus(P_1)^{u+\delta_0}(a, u, \sigma) \cup \ominus(P_2)^{u+\delta_0}(a, u, \sigma))$.

Nondeterministic Choice. $P_1 \parallel P_2$ is the usual alternative composition, only one alternative is chosen nondeterministically, the operational semantics is:

$$\frac{\langle P_i, (a, u, \sigma) \rangle \rightarrow \mathbf{P}_i \quad (i \in \{1, 2\})}{\langle P_1 \parallel P_2, (a, u, \sigma) \rangle \rightarrow \mathbf{P}_i}.$$

If-Then. When the guard B is true under environment (a, u, σ) , agent behaves as process P :

$$\frac{\langle B, (a, u, \sigma) \rangle \rightarrow \mathbf{P} \wedge \mathfrak{T}(B)(a, u, \sigma) = T}{\langle B \rightarrow P, (a, u, \sigma) \rangle \rightarrow \mathbf{P} \circ \mathbf{M}_\rightarrow^{-1}}$$

where $\mathbf{M}_\rightarrow(\langle as, \langle E, e \rangle \rangle) \stackrel{def}{=} \langle as, \langle P, e \rangle \rangle$, and $e = (\iota(B)(a, u, \sigma), t(B) + \tau(B), \sigma)$.

Interruption. Timeout interruption $P_1 \supseteq_\delta P_2$ will behave like P_1 before δ time units and then as P_2 . When the process executes $\delta_0 \leq \delta$ time units:

$$\frac{\langle P_1, (a, u, \sigma) \rangle \rightarrow \mathbf{P}}{\langle P_1 \supseteq_\delta P_2, (a, u, \sigma) \rangle \rightarrow \mathbf{P} \circ \mathbf{M}_{\supseteq_\delta}^{-1}}$$

where

$$\mathbf{M}_{\supseteq_\delta}(\langle as, \langle P'_1, e \rangle \rangle) \stackrel{def}{=} \begin{cases} \langle as, \langle P'_1 \supseteq_{\delta-\delta_0} P_2, e \rangle \rangle & \text{if } \delta_0 < \delta \\ \langle as, \langle P_2, e \rangle \rangle & \text{if } \delta_0 = \delta \end{cases}$$

where $e = (\iota(P_1)^{u+\delta_0}(a, u, \sigma), u + \delta_0, \kappa(P_1)^{u+\delta_0}(a, u, \sigma))$.

Let

$$Q_i \equiv \text{Get}_{(l,t)}^{\leftarrow G'}(m_i, \delta)$$

Get interruption $P \supseteq \llbracket_{i \in I} \text{Get}_{(l,t)}^{\leftarrow G'}(m_i, \delta) \rightarrow P_i$ behaves like P_1 until the agent receives one message m_i successfully, then it behaves as P_i . When get interruption process executes δ_0 time units:

$$\frac{\langle P, (a, u, \sigma) \rangle \rightarrow \mathbf{P}_1 \wedge \langle Q_i, (l, t, \sigma') \rangle \rightarrow \mathbf{P}_2 \wedge m_{iG' \uparrow G} \in \sigma_{G'}}{\langle P \supseteq \llbracket_{i \in I} Q_i \rightarrow P_i, (a, u, \sigma) \rangle \rightarrow (\mathbf{P}_1 \times \mathbf{P}_2) \circ \mathbf{M}_{\supseteq_{Get}}^{-1}}$$

where $\sigma' = \kappa(P)^t(a, u, \sigma)$ and

$$\mathbf{M}_{\supseteq_{Get}}(\langle as_1, \langle P', e_1 \rangle \rangle, \langle as_2, \langle Q'_i, e_2 \rangle \rangle) \stackrel{def}{=} \begin{cases} \langle as_1 \cup as_2, \langle P' \supseteq Q_i \rightarrow P_i, e \rangle \rangle & \text{if } u + \delta_0 < t \\ \langle as_1 \cup as_2, \langle P' \supseteq Q'_i \rightarrow P_i, e \rangle \rangle & \text{if } u + \delta_0 \geq t \text{ and } Q'_i \neq E \\ \langle as_1 \cup as_2, \langle P_i, e \rangle \rangle & \text{if } u + \delta_0 \geq t \text{ and } Q'_i = E \end{cases}$$

where $e_1 = (l_1, u + \delta_0, \sigma_1)$, $e_2 = (l_2, u + \delta_0, \sigma_2)$, $e = (\text{Last}\{l_1, l_2\}, u + \delta_0, \sigma_{\supseteq_{Get}})$, in which $l_1 = \iota(P)^{u+\delta_0}(a, u, \sigma)$, $\sigma_1 = \kappa(P)^{u+\delta_0}(a, u, \sigma)$, $l_2 = \iota(Q_i)^{u+\delta}(l, t, \sigma')$, $\sigma_2 = \kappa(Q_i)^{u+\delta_0}(l, t, \sigma')$, $\sigma_{\supseteq_{Get}} = \sigma_1 \cup \sigma_2 \setminus (\ominus(P)^{u+\delta_0}(a, u, \sigma) \cup \ominus(Q_i)^{u+\delta_0}(l, t, \sigma'))$.

Probabilistic Choice. Process $C \rightarrow +_{i \in I} p_i : \{as_i\}$ P_i first behaves as C and when C terminates successfully it randomly selects an alternative $i \in I$ according to the probability p_i , performs an assignment according to as_i , and continues executing P_i :

$$\frac{(a, u) = (l(C), t(C)) \wedge \mathfrak{T}(C)(a, u, \sigma) = T}{\langle C \rightarrow +_{i \in I} p_i : \{as_i\} P_i, (a, u, \sigma) \rangle \rightarrow \mathbf{P}}$$

where $\mathbf{P}(\langle as_i, \pi_i \rangle) = p_i$, in which π_i stands for the i th configuration with i th process P_i and i th environment e_i (i.e., $\pi_i = \langle P_i, e_i \rangle$), note that $\sum_{i \in I} p_i = 1$.

Operational Semantics for System Process. When different agents behave concurrently in the system, they have their own traces of location and storages but the same global time. The concurrent operation between two processes will be like handshaking over the interactions (i.e., Send and Get) or behave independently otherwise. Similarly, unlike interleaving, since each process has to follow the specification, the operational semantics for the system process is actually combined by the operational semantics of sub agent processes with checking whether the handshaking is legal (i.e., $\text{Send}_{(l,t)}^{\rightarrow G_1}(m, \delta)$ and $\text{Get}_{(l,t)}^{\leftarrow G_2}(m, \delta)$ in P_{G_2} and P_{G_1} respectively with the same triggered time, duration and message) if there exists one. We define a configuration $\mathbf{Stconf}_S$ for a system process as $\mathcal{S} \times \mathcal{E}^n$, where n stands for the number of sub agent processes in S , a system configuration of the form $\langle S, EN^n \rangle$ is denoted by π_S , where $S = P_{G_1} \bowtie P_{G_2} \bowtie \dots \bowtie P_{G_n}$ and $EN^n = (e_1, e_2, \dots, e_n)$. We use $\pi_{S_i} = \langle P_{G_i}, e_i \rangle$ to denote the configuration for i th sub agent process, where $e_i = (a_i, u, \sigma_i)$ (they have the same u since it stands for the global time).

Definition 2. The operational semantics for system process S with n sub agent processes is defined as a subset $\rightarrow_S \subseteq \mathbf{Stconf}_S \times \mathbf{P}_S(\mathbb{A}^n \times \mathbf{Stconf}_S)$.

where $\mathbf{P}_S : \mathbb{A}^n \times \mathbf{Stconf}_S \rightarrow [0, 1]$ is a function to map n -element set of assignments and n -element configurations onto a probability.

For $\langle \pi_S, \mathbf{P}_S(AS^n, \pi'_S) \rangle \in \rightarrow_S$, where $AS^n = (as_1, as_2, \dots, as_n)$, we write $\pi_S \rightarrow_S \mathbf{P}_S(AS^n, \pi'_S)$, where $\pi_{S_i} \rightarrow \mathbf{P}(as_i, \pi'_{S_i})$. Once $\pi_S \rightarrow_S \mathbf{P}_S(AS^n, \pi'_S)$ is executed, the system configuration is changed to π'_S with sub agent process P_{G_i} assigning its variables values according to as_i with probability $\mathbf{P}_S(\langle AS^n, \pi'_S \rangle) = \prod_{i \in n} \mathbf{P}(\langle as_i, \pi'_{S_i} \rangle)$. Particularly, the operational semantics of system process with one agent process is same as that of the latter, i.e., we simply rewrite $\rightarrow_S$ as $\rightarrow$. The operational semantics for system process S with $n(\geq 2)$ sub agent processes $P_{G_1}, \dots, P_{G_n}$ is presented as follows:

$$\frac{\langle P_{G_1}, e_1 \rangle \rightarrow \mathbf{P}_{G_1} \wedge \dots \wedge \langle P_{G_n}, e_n \rangle \rightarrow \mathbf{P}_{G_n} \wedge \mathbf{CheckLegal}}{\langle P_{G_1} \bowtie \dots \bowtie P_{G_n}, EN^n \rangle \rightarrow_S (\mathbf{P}_{G_1} \times \dots \times \mathbf{P}_{G_n}) \circ \mathbf{M}_{\bowtie}^{-1}}$$

where

$$\mathbf{M}_{\bowtie}(\langle as_1, \langle P'_{G_1}, e'_1 \rangle \rangle, \dots, \langle as_n, \langle P'_{G_n}, e'_n \rangle \rangle) \stackrel{def}{=} \langle AS^n, \langle P'_{G_1} \bowtie \dots \bowtie P'_{G_n}, EN^{n'} \rangle \rangle$$

and **CheckLegal** is a function to check whether the interactions are legal in S , i.e., for each pair $\text{Send}_{(l_i, t_i)}^{\rightarrow G_j}(m_i, \delta_i)$ and $\text{Get}_{(l_j, t_j)}^{\leftarrow G_i}(m_j, \delta_j)$ in P_{G_i} and P_{G_j} respectively with $t_i = t_j$, if $m_i = m_j$ and $\delta_i = \delta_j$, then it returns **True**, else returns **False**. If no such pair exists, **CheckLegal** returns **True**.

3 Automatic Spray Painting of a Custom Car

Here we consider one smart spray painting factory for custom cars using Internet of Things techniques. The whole factory is ‘alive’ that every part

(i.e., physical thing) of the system is able to sense the environment, communicate with each other and react in real time. Once a car is placed at the door of factory, it has been loaded with all its booking information, including the car number, the customer name, the timestamp of each event, the map and the detailed spray painting requirements (e.g., patterns, colors and paint material). The car moves automatically following the map. When starting approach the robots at location *Lappr*, it sends all necessary information (*AP*) to the robots. After receiving the information, the robots check the store of all paint material. They would find lack of some material with probability p_{fail} . Then the robots send message *NE* (which is ‘do Not Enter’) to forbid the car to enter the workstation. When the car receives the message at location *Lpass*, it stops progressively to location *Lstop*. Meanwhile the robots automatically refill the material, then send message *ET* (which is ‘EnTer’) to allow the car to enter. After receiving the message, the car continues to move towards the robots. When the automatic spray painting finishes, the robots send message *OK* to the car. Then the car moves to location *Lleav* to end its procedure. We first establish two agent processes **Car** and **Robot**, with messages $\{AP, NE, ET, OK\}$, actions $\{\text{Send, Get, Run, Stopping, Appr, Refill, Check}\}$, states $\{\text{Idle, Painting}\}$, and locations $\{Lappr, Lpass, Lstop, Lleav\}$. We assume the message transmitting time is 2 unit time.

The agent process **Car** is defined as:

$$(\text{Send}_{(Lappr,t)}^{\rightarrow R}(AP, 2) \parallel \text{Appr}_{(Lappr,t)}(Lpass, \delta_1)) \geq \left(\begin{array}{l} \text{Get}_{(Lpass,t+\delta_1-2)}^{\leftarrow R}(ET, 2) \rightarrow \text{Appr}_{(Lpass,t+\delta_1)}(L, \delta_2); \\ \quad \text{Wait}_{(L,t+\delta_1+\delta_2)}^{\leftarrow R}(OK, \delta_3); \text{Run}_{(L,t+\delta_1+\delta_2+\delta_3)}(Lleav, \delta_4) \rightarrow \text{Stop} \\ \parallel \\ \text{Get}_{(Lpass,t+\delta_1-2)}^{\leftarrow R}(NE, 2) \rightarrow \text{Stopping}_{(Lpass,t+\delta_1)}(Lstop, \delta_5); \\ \quad \text{Wait}_{(Lstop,t+\delta_1+\delta_5)}^{\leftarrow R}(ET, \delta_6); \text{Appr}_{(Lstop,t+\delta_1+\delta_5+\delta_6)}(L, \delta_7); \\ \quad \text{Wait}_{(L,t+t_1)}^{\leftarrow R}(OK, \delta_8); \text{Run}_{(L,t+t_1+\delta_8)}(Lleav, \delta_9) \rightarrow \text{Stop} \end{array} \right)$$

where $t_1 = \delta_1 + \delta_5 + \delta_6 + \delta_7$.

The agent process **Robot** is defined as:

$$\text{Idle}_{(L,t_0)}(\infty) \geq \begin{array}{l} \text{Get}_{(L,t)}^{\leftarrow C}(AP, 2) \rightarrow \text{Check}_{(L,t+2)}(L, \theta_1) \rightarrow \\ \left(\begin{array}{l} 0.8 : \{st = satisfied\} \\ \quad (\text{Send}_{(L,t+2+\theta_1)}^{\leftarrow C}(ET, 2) \parallel \text{Wait}_{(L,t+2+\theta_1)}(\theta_2)) \geq \theta_2 \\ \quad \text{Painting}_{(L,t+2+\theta_1+\theta_2)}(\theta_3); \text{Send}_{(L,t+t_2)}^{\leftarrow C}(OK, 2); \\ \quad \text{Idle}_{(L,t+t_2+2)}(\infty) \\ + \\ 0.2 : \{st = lack\} \\ \quad (\text{Send}_{(L,t+2+\theta_1)}^{\leftarrow C}(NE, 2) \parallel \text{Refill}_{(L,t+2+\theta_1)}(L, \theta_4)); \\ \quad (\text{Send}_{(L,t+2+\theta_1+\theta_4)}^{\leftarrow C}(ET, 2) \parallel \text{Wait}_{(L,t+2+\theta_1+\theta_4)}(\theta_5)) \geq \theta_5 \\ \quad \text{Painting}_{(L,t+t_3)}(\theta_6); \text{Send}_{(L,t+t_3+\theta_6)}^{\leftarrow C}(OK, 2); \\ \quad \text{Idle}_{(L,t+t_3+\theta_6+2)}(\infty) \end{array} \right) \end{array}$$

where variable st stands for the status of required material with initial value *satisfied*, and $t_2 = 2 + \theta_1 + \theta_2 + \theta_3$, $t_3 = 2 + \theta_1 + \theta_4 + \theta_5$.

The PSTeC model for the smart spray painting factory is specified as the system process: **Car** $\bowtie$ **Robot**, and we restrict that $t + \delta_1 - 2 = t + 2 + \theta_1$, $t + \delta_1 + \delta_2 + \delta_3 = t + t_2 + 2$, $t + \delta_1 + \delta_5 + \delta_6 = t + 2 + \theta_1 + \theta_4 + 2$, $t + t_1 + \delta_8 = t + t_3 + \theta_6 + 2$ for each pair of handshaking interactions.

4 Conclusion

In this paper, we have developed a modelling formalism, PSTeC, for specification of probabilistic real-time systems triggered by location and time. For this, we have extended the spatial-temporal consistency language STeC by adding probabilistic operations so that PSTeC, apart from location specifics, supports the specification of nondeterministic, real-time and probabilistic aspects of behaviours. Considering concurrency between agents is far different from behaviours of one agent since sub agent processes in one system process have different traces of location and different message storages. We defined the syntax of PSTeC for system and agent separately, with some useful shorthands. Then we provided full details of the operational semantics, using the structured operational semantics style. After establishing the formalism of PSTeC, we refined the STeC model of a smart spray painting factory with additional probabilistic operations reflecting the existing probabilistic behaviours.

Recently, we have been working on a denotational semantics for PSTeC aiming at soundness as well as completeness. A tool to support building PSTeC models and doing verification is under development.

Acknowledgments. This work is supported by the National Basic Research Program of China (Grant No. 2011CB302802), the Innovation Group Project of the National Natural Science Foundation (Grant No. 61321064), the National Natural Science Foundation of China (Grant No. 61370100), the NSFC projects (Grant No. 61361136002 and No. 61202105) and Shanghai Knowledge Service Platform for Trustworthy Internet of Things (Grant No. ZF1213). Part of this work was done while the first author was visiting Saarland University, Germany. The authors thank Holger Hermanns (Saarland University, Germany) for his valuable contributions and discussions and helpful comments on the structure and contents of this paper. The authors would also like to thank the anonymous referees for their invaluable comments and suggestions.

References

1. Bohnenkamp, H., DArgenio, P.R., Hermanns, H., Katoen, J.: Modest: a compositional modeling formalism for hard and softly timed systems. *IEEE Trans. Softw. Eng.* **32**(10), 812–830 (2006)
2. Chen, Y.: STeC: a location-triggered specification language for real-timesystems. In: 2012 15th IEEE International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing Workshops (ISORCW), pp.1–6. IEEE (2012)

3. Chen, Y., Wu, H.: Semantics of sub-probabilistic programs. *Front. Comput. Sci. China* **2**(1), 29–38 (2008)
4. Chen, Y., Zhang, Y.: A hybrid clock system related to STeC language. In: 2014 IEEE Eighth International Conference on Software Security and Reliability-Companion (SRE-C), pp. 199–203. IEEE (2014)
5. Dijkstra, E.W.: *A Discipline of Programming*, vol. 1. Prentice-Hall, Englewood Cliffs (1976)
6. Hahn, E.M., Hartmanns, A., Hermanns, H., Katoen, J.: A compositional modelling and analysis framework for stochastic hybrid systems. *Formal Methods Syst. Des.* **43**(2), 191–232 (2013)
7. Hahn, E.M., Hermanns, H., Wachter, B., Zhang, L.: INFAMY: an infinite-state markov model checker. In: Bouajjani, A., Maler, O. (eds.) CAV 2009. LNCS, vol. 5643, pp. 641–647. Springer, Heidelberg (2009)
8. He, K., Zhang, M., He, J., Chen, Y.: Probabilistic model checking of pipe protocol. In: 2015 International Symposium on Theoretical Aspects of Software Engineering (TASE), pp. 135–138. IEEE (2015)
9. Hermanns, H., Katoen, J.P., Meyer-Kayser, J., Siegle, M.: ETMCC: model checking performability properties of markov chains. In: Null, p. 673. IEEE (2003)
10. Jifeng, H., Seidel, K., McIver, A.: Probabilistic models for the guarded command language. *Sci. Comput. Program.* **28**(2), 171–192 (1997)
11. Katoen, J., Zapreev, I.S., Hahn, E.M., Hermanns, H., Jansen, D.N.: The ins and outs of the probabilistic model checker MRMC. *Perform. Eval.* **68**(2), 90–104 (2011)
12. Kwiatkowska, M., Norman, G., Parker, D.: PRISM 4.0: verification of probabilistic real-time systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) CAV 2011. LNCS, vol. 6806, pp. 585–591. Springer, Heidelberg (2011)
13. Kwiatkowska, M., Norman, G., Segala, R., Sproston, J.: Automatic verification of real-time systems with discrete probability distributions. *Theor. Comput. Sci.* **282**(1), 101–150 (2002)
14. Kwiatkowska, M., Norman, G., Sproston, J., Wang, F.: Symbolic model checking for probabilistic timed automata. *Inf. Comput.* **205**(7), 1027–1077 (2007)
15. Plotkin, G.D.: *A structural approach to operational semantics* (1981)
16. Segala, R., Lynch, N.: Probabilistic simulations for probabilistic processes. In: Jonsson, B., Parrow, J. (eds.) CONCUR 1994. LNCS, vol. 836, pp. 481–496. Springer, Heidelberg (1994)
17. Shiryaev, A.N.: *Probability*. Graduate Texts in Mathematics, vol. 95 (1996)
18. Wu, H., Chen, Y., Zhang, M.: On denotational semantics of spatial-temporal consistency language-stec. In: 2013 International Symposium on Theoretical Aspects of Software Engineering (TASE), pp. 113–120. IEEE (2013)
19. Ying, M.: Reasoning about probabilistic sequential programs in a probabilistic logic. *Acta Informatica* **39**(5), 315–389 (2003)